

# Large Language Models en Sistemas Inteligentes

## Herramientas y Aplicaciones de la Inteligencia Artificial

Objetivo: Toma de decisiones en entornos complejos

01

---

### Comprender los LLM

Qué son los Large Language Models y cómo funcionan internamente

02

---

### LLM en sistemas de IA

Su papel como herramienta dentro de arquitecturas inteligentes

03

---

### Toma de decisiones

Uso de LLM para razonamiento y decisión en entornos complejos

04

---

### Agentes inteligentes

Integración de LLM en agentes que actúan sobre entornos dinámicos



# Contexto: herramientas en sistemas de IA

Los sistemas modernos de Inteligencia Artificial rara vez se basan en un único enfoque. En la práctica, combinan múltiples técnicas complementarias para resolver problemas de distintas naturalezas con mayor eficacia y robustez.

## **Heurísticas y reglas**

Conocimiento experto codificado explícitamente

## **Planificación**

Algoritmos de búsqueda y secuenciación de acciones

## **Optimización**

Maximización de objetivos bajo restricciones

## **Aprendizaje automático**

Modelos que aprenden patrones a partir de datos

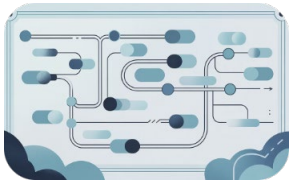
## **Modelos generativos**

Generación de contenido y razonamiento flexible

 En este contexto, los **Large Language Models** se han convertido en una nueva herramienta que puede incorporarse de forma natural a sistemas existentes, ampliando sus capacidades de razonamiento y comprensión.

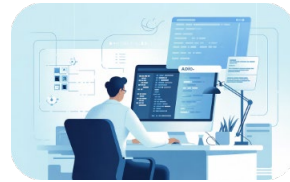
# Los LLM como componente de sistemas inteligentes

Aunque popularmente se asocian con chatbots y asistentes conversacionales, los LLM ofrecen capacidades mucho más amplias. Pueden actuar como **módulos de razonamiento** dentro de arquitecturas de IA más complejas, realizando tareas cognitivas que antes requerían soluciones ad hoc.



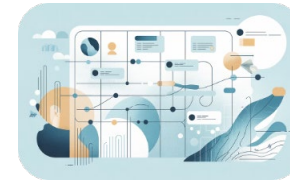
## Razonamiento complejo

Análisis y comparación de múltiples alternativas en problemas no estructurados



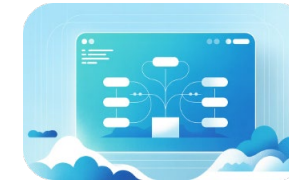
## Generación de código

Producción automática de soluciones programáticas a partir de especificaciones en lenguaje natural



## Planificación de acciones

Descomposición de objetivos en secuencias de pasos ejecutables



## Toma de decisiones

Selección informada de acciones bajo información estructurada o ambigua

# ¿Qué es un Large Language Model?

Un Large Language Model es un modelo probabilístico entrenado para predecir la siguiente unidad de texto —llamada **token**— dada una secuencia de contexto previa.

Formalmente, el modelo aprende una distribución de probabilidad condicional:

$$P(\text{next\_token} \mid \text{context})$$

A través de entrenamiento masivo sobre grandes corpus de texto, el modelo internaliza representaciones complejas del lenguaje, la sintaxis, la semántica y el conocimiento implícito en los datos.



## Arquitectura base

Transformers con mecanismo de atención multi-cabeza (multi-head attention). Se fija en **distintos tipos de relaciones al mismo tiempo** dentro de una secuencia

## Entrenamiento

Predicción del siguiente token sobre billones de ejemplos de texto

## Resultado

Representaciones ricas del lenguaje y conocimiento del mundo



# Escalado y capacidades emergentes

A medida que se incrementan los recursos de entrenamiento —parámetros, datos y cómputo— los modelos exhiben **capacidades emergentes**: habilidades que no estaban presentes en modelos más pequeños y que no se obtienen por extrapolación lineal.

**~1T**  
10<sup>11</sup> – 10<sup>12</sup> parámetros

## Parámetros

Los modelos más avanzados alcanzan escala de billones (trillones anglosajones) de parámetros entrenables. Solemos ver los de billones anglosajones, por ejemplo, llama3.3:70b

**10T+**

## Tokens de entrenamiento

Volumen de texto procesado durante el preentrenamiento de modelos de frontera

### Razonamiento paso a paso

Chain-of-thought y descomposición de problemas complejos



### Generación de código

Síntesis de programas correctos en múltiples lenguajes



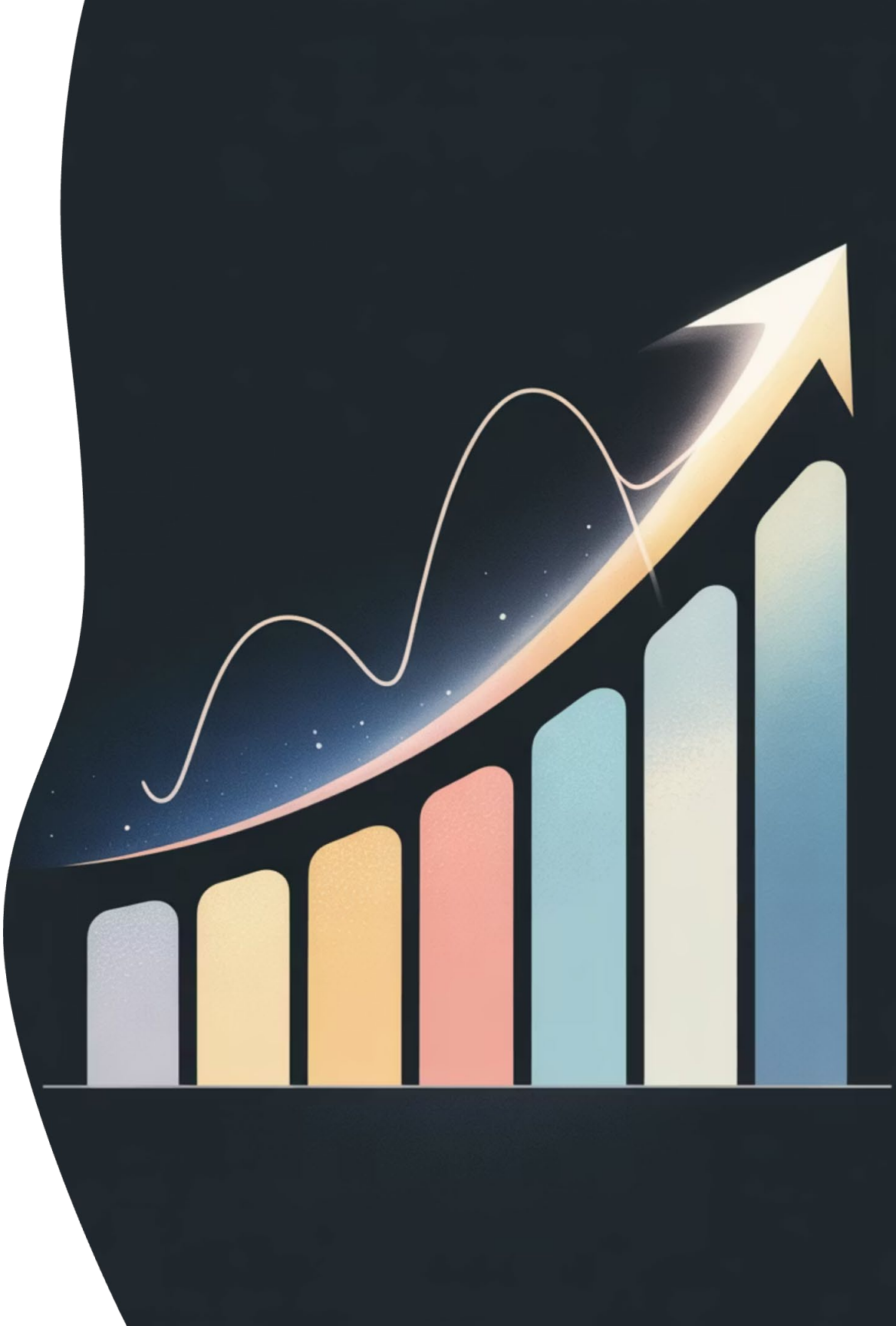
### Análisis de problemas

Evaluación crítica y comparación de alternativas complejas



### Planificación

Generación de planes de acción coherentes y secuenciados



# Limitaciones computacionales

El uso de LLM introduce una cadena de costes computacionales que deben considerarse en el diseño del sistema. Cada llamada al modelo implica un conjunto de operaciones con coste no despreciable:

## → Construcción del prompt

Formateo y enriquecimiento del contexto antes de la llamada

## → Ejecución de la inferencia

Paso forward a través de las capas del modelo

## → Generación de la respuesta

Decodificación autoregresiva token a token

## → Interpretación de la salida

Parseo y validación del resultado generado

### ⚠ Factor crítico: latencia

En sistemas donde se toman **muchas decisiones consecutivas**, el tiempo de respuesta acumulado puede convertirse en un cuello de botella operativo.

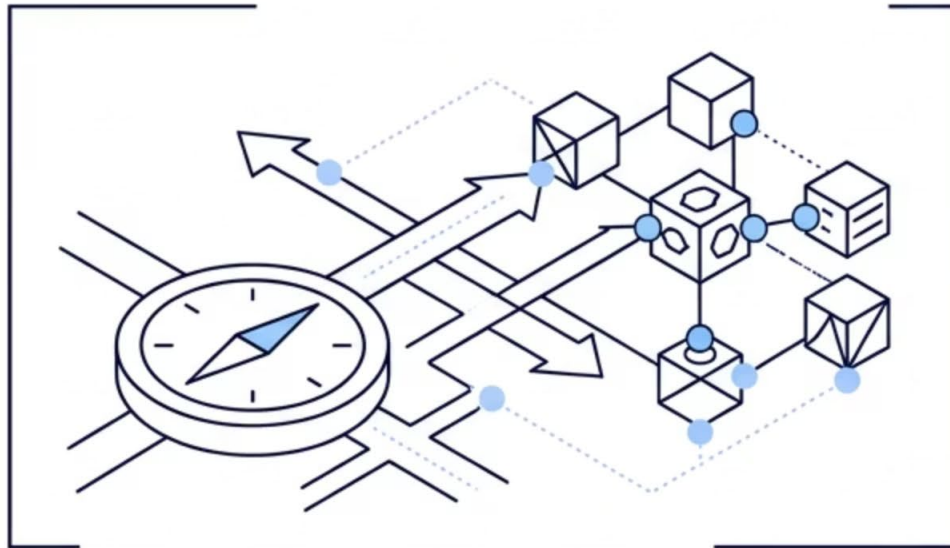
La latencia total depende de:

- Tamaño del modelo (nº de parámetros)
- Longitud del contexto procesado
- Hardware disponible (GPU/CPU)
- Congestión de la red o API

# Selección de decisiones donde usar LLM

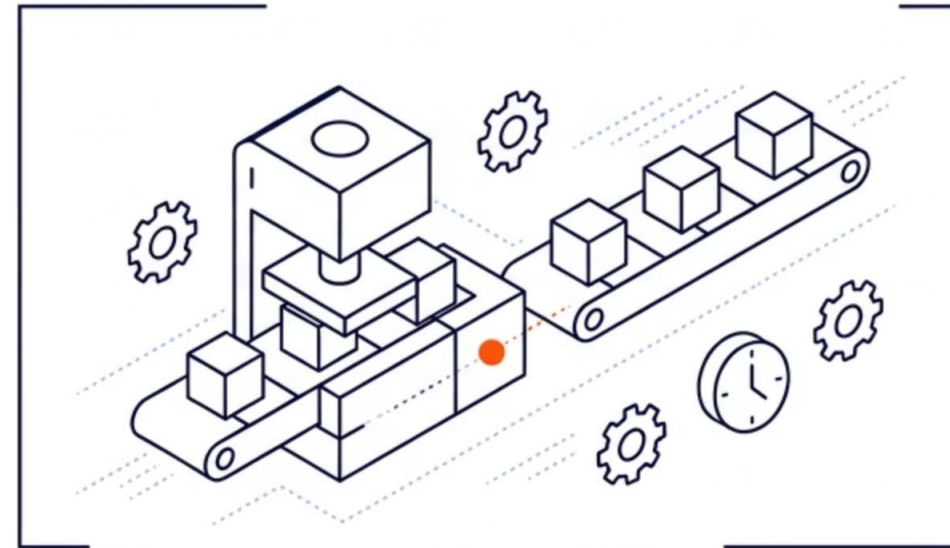
En sistemas reales, el uso indiscriminado de LLM resulta ineficiente. La buena práctica consiste en evaluar cuidadosamente **qué decisiones justifican el coste** de una llamada al modelo y cuáles pueden resolverse con mecanismos más ligeros.

## MÁS ADECUADO



**decisiones estratégicas de alto nivel**  
**evaluación de múltiples alternativas**  
**complejas**  
**razonamiento contextual con**  
**información ambigua**

## MENOS ADECUADO



**decisiones triviales o repetitivas**  
**acciones extremadamente frecuentes**  
**de bajo coste computacional**  
**contextos con requisitos estrictos**  
**de latencia en tiempo real**





# Opciones de acceso a modelos

La elección del modo de acceso al LLM es una decisión de arquitectura con implicaciones directas sobre el rendimiento, el coste y la privacidad del sistema.

1

## Ejecución local

Modelos abiertos ejecutados en hardware propio. Máximo control, sin dependencias externas.

- Control total del entorno de ejecución
- Sin coste por inferencia
- Privacidad de los datos garantizada

2

## APIs institucionales

Acceso a modelos de frontera mediante APIs gestionadas por instituciones o proveedores especializados.

- Modelos de mayor capacidad
- Sin necesidad de hardware propio
- Coste proporcional al uso

3

## Servicios cloud

Plataformas como Azure OpenAI, Google Vertex AI o AWS Bedrock que ofrecen LLM como servicio escalable.

- Alta disponibilidad y escalabilidad
- Integración sencilla con otros servicios
- Dependencia de terceros



# Modelos ejecutados en local

La ejecución local de modelos abiertos representa una opción estratégica para entornos de investigación y desarrollo donde el control del entorno y la reproducibilidad son prioritarios. Permite utilizar LLM sin depender de servicios externos ni incurrir en costes por inferencia.



## Control total

Sin dependencias externas ni transferencia de datos



## Sin coste por uso

Inferencia ilimitada sobre hardware disponible



## Menor latencia de red

Sin llamadas remotas ni colas de API externas



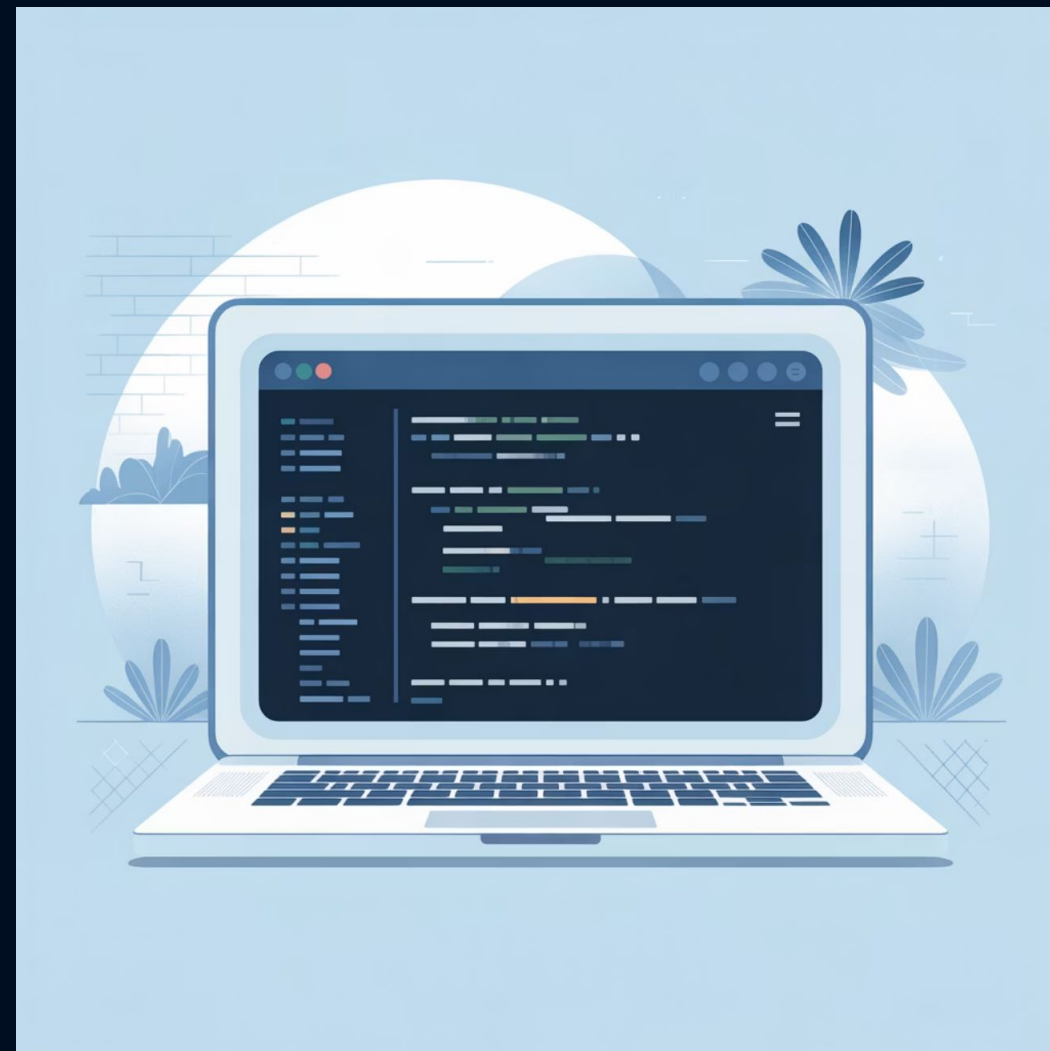
## Reproducibilidad

Resultados consistentes en experimentos y evaluaciones

### Ollama

Ollama es actualmente una de las herramientas más utilizadas para ejecutar modelos de lenguaje abiertos en local. Ofrece una interfaz sencilla para descargar, gestionar y ejecutar modelos como **LLaMA**, **Mistral** o **Gemma** con un único comando.

Ollama abstrae la complejidad de la configuración de entornos GPU/CPU, facilitando la integración de LLM en flujos de trabajo de investigación y desarrollo.



# API Institucional de la UPV

La Universitat Politècnica de València ofrece acceso a múltiples modelos a través de [api.poligpt.upv.es](https://api.poligpt.upv.es), sin necesidad de ejecutarlos localmente.



## Modelos ligeros

llama3.1:8b, qwen3:4b, phi4-mini:3.8b



## Modelos medianos

llama, qwen, gemma, phi4



## Modelos avanzados

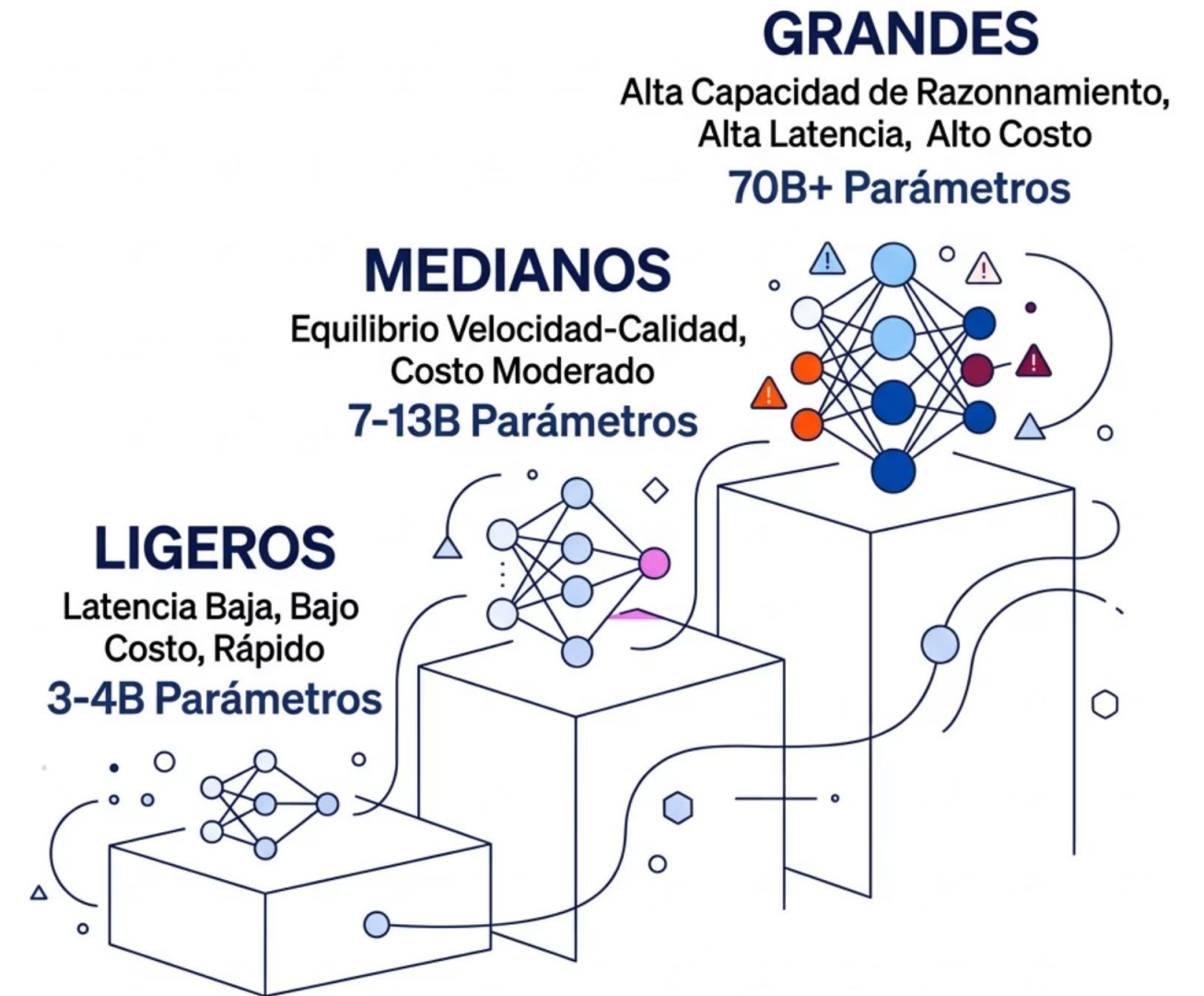
deepseek-reasoner (r1) — capacidad de razonamiento



# Categorías de Modelos

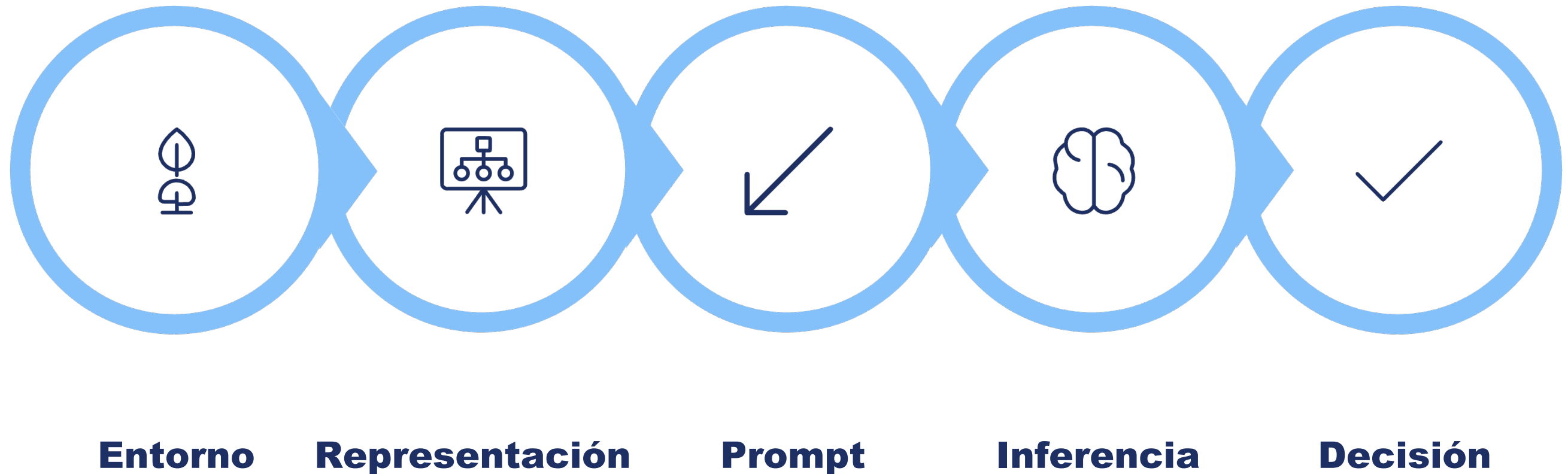
La elección del modelo adecuado depende del contexto de uso. No siempre el modelo más potente es la mejor opción: la latencia y el coste son factores críticos en sistemas en tiempo real.

📌 El equilibrio entre capacidad y eficiencia es clave para la viabilidad del sistema.



# Integración de un LLM en un Agente

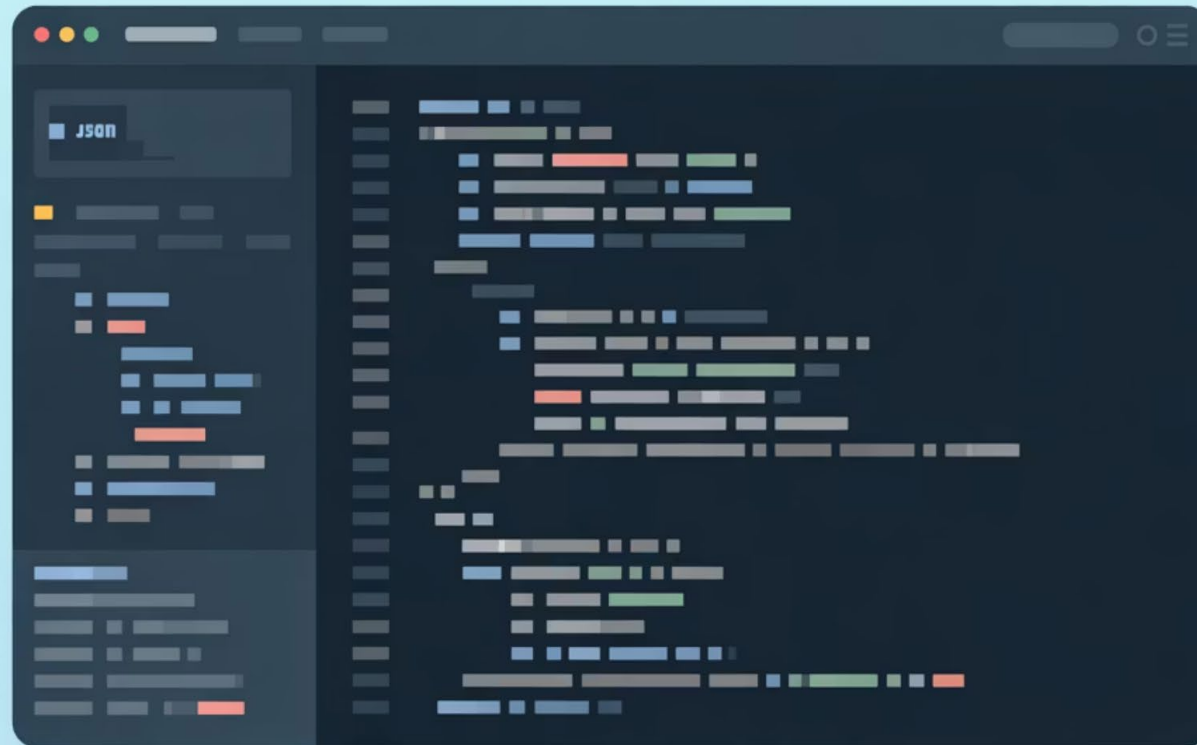
Un modelo de lenguaje actúa como módulo de decisión dentro de un sistema mayor. El flujo de información sigue una cadena bien definida, desde la observación del entorno hasta la ejecución de una acción concreta.



Esta arquitectura permite desacoplar la lógica de decisión del resto del sistema, facilitando la sustitución o actualización del modelo subyacente.



# Representación Estructurada de la Información



## ¿Por qué usar JSON?

Al integrar un LLM en software, es fundamental que las respuestas sean procesables de forma automática. El formato JSON permite representar de manera precisa:

### Estado del sistema

Contexto actual del entorno

### Opciones disponibles

Alternativas válidas para el agente

### Decisión final

Salida parseada directamente en código

# Ejemplo de Salida Estructurada

## Respuesta esperada del modelo

El agente solicita al LLM una decisión en formato JSON. La respuesta puede interpretarse directamente dentro del código sin procesamiento adicional:

```
{ "node_id": 12, "road_to": 18 }
```

Este enfoque elimina la necesidad de parsear texto libre y reduce significativamente los errores de integración.

## Ventajas del formato estructurado

- Parseo directo sin expresiones regulares
- Validación de esquema con herramientas estándar
- Integración limpia con APIs REST y sistemas de mensajería



# Diseño del Prompt

La calidad del prompt determina en gran medida el comportamiento del modelo. Un prompt bien estructurado mejora la consistencia y reduce la latencia evitando información superflua.



## Instrucciones claras

Define la tarea sin ambigüedad



## Contexto relevante

Solo la información necesaria para decidir



## Formato de salida

Especifica el esquema JSON esperado



## Alternativas válidas

Lista acotada de opciones para el modelo



Un prompt excesivamente largo puede incrementar la latencia sin mejorar la calidad de la decisión.

# Diseño de prompts para toma de decisiones en agentes (PyCatan)

## Objetivo del prompt

El prompt debe permitir al modelo:

- entender el estado del entorno
- evaluar un conjunto de acciones válidas
- seleccionar una acción coherente y válida

El LLM actúa como **selector de decisiones**, no como generador libre.

## Estructura recomendada del prompt

### 1.Contexto / rol del agente

Definir claramente el comportamiento esperado  
(ej. “Eres un agente que juega a Catan...”)

### 2.Estado del entorno (resumido)

Incluir solo la información relevante  
(recursos, posiciones, probabilidades, etc.)

### 3.Acciones candidatas

Lista cerrada de opciones válidas  
(evitar generación abierta)

### 4.Formato de salida

Especificar salida estructurada (JSON o equivalente)

## Ejemplo conceptual (simplificado)

Entrada:

- lista de nodos candidatos
- puntuaciones heurísticas
- posibles carreteras

Instrucción:

“Selecciona el mejor nodo y carretera entre los candidatos.  
Responde únicamente con un objeto estructurado.”

Salida esperada:

node\_id: X

road\_to: Y

## Buenas prácticas

- limitar el número de candidatos (top-k)
- usar formato estructurado (JSON)
- evitar texto innecesario
- dar instrucciones claras y cerradas
- validar siempre la salida del modelo

## Problemas comunes

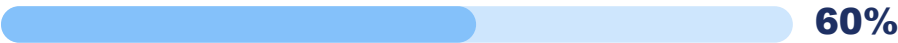
- prompts demasiado largos → alta latencia
- demasiada información → peor decisión
- salida no estructurada → errores en parsing
- decisiones inconsistentes entre ejecuciones



# Tokens y Eficiencia

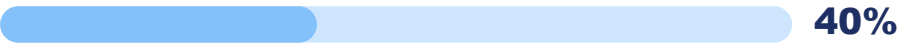
## ¿Qué son los tokens?

Los LLMs procesan el texto en unidades llamadas **tokens** —aproximadamente 0,75 palabras en inglés. El número de tokens en cada llamada afecta directamente al rendimiento y al coste del sistema.



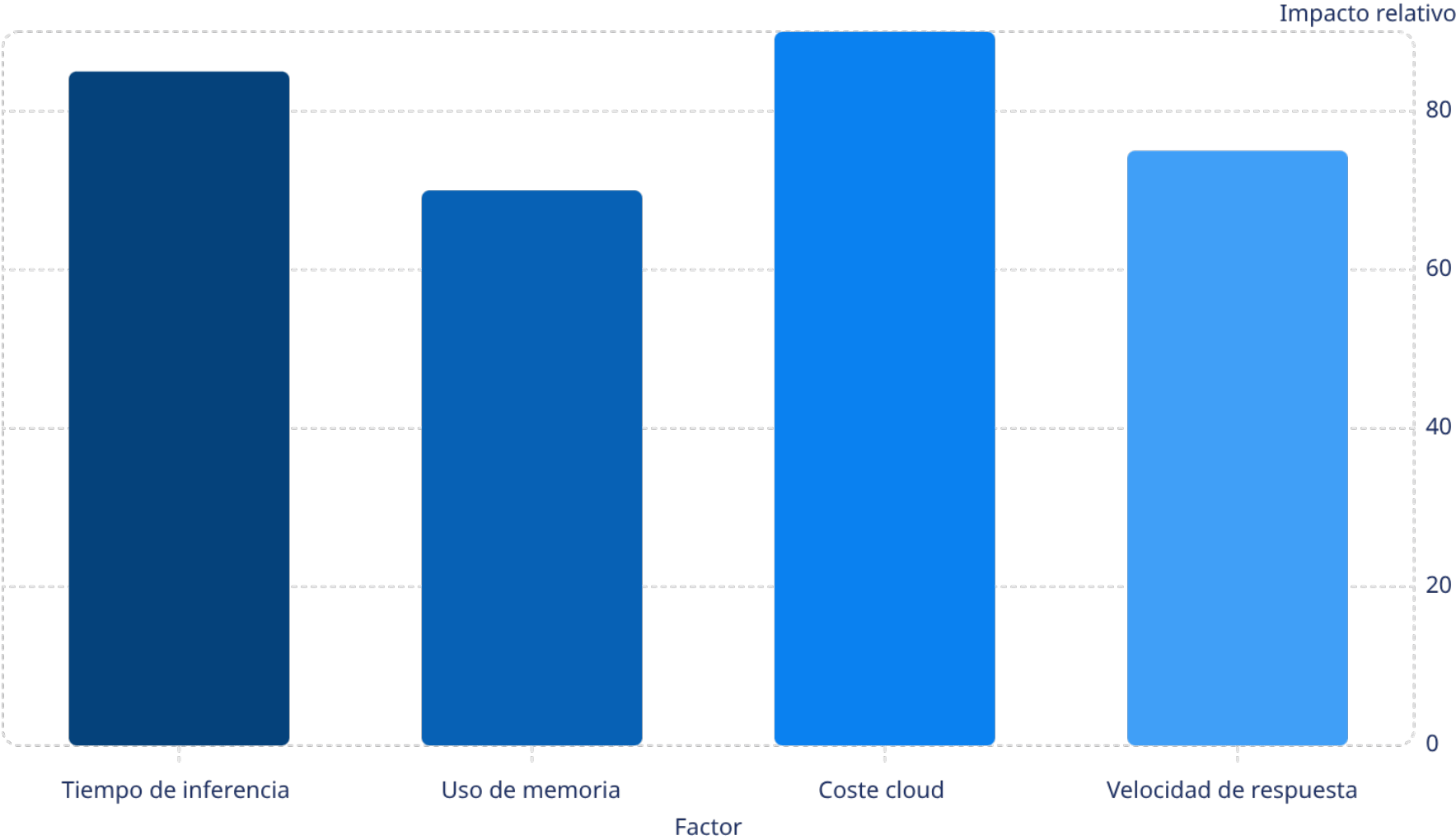
### Reducción de latencia

al eliminar contexto redundante



### Ahorro de coste

limitando alternativas evaluadas



Impacto del volumen de tokens en los factores clave del sistema.

# Evaluación de Modelos

Seleccionar el modelo adecuado requiere analizar su comportamiento dentro del sistema completo, no solo la calidad aislada de sus respuestas.

1

2

## Tiempo de respuesta

Latencia end-to-end en condiciones reales de carga

## Calidad de decisiones

Precisión y coherencia frente a distintos escenarios

3

4

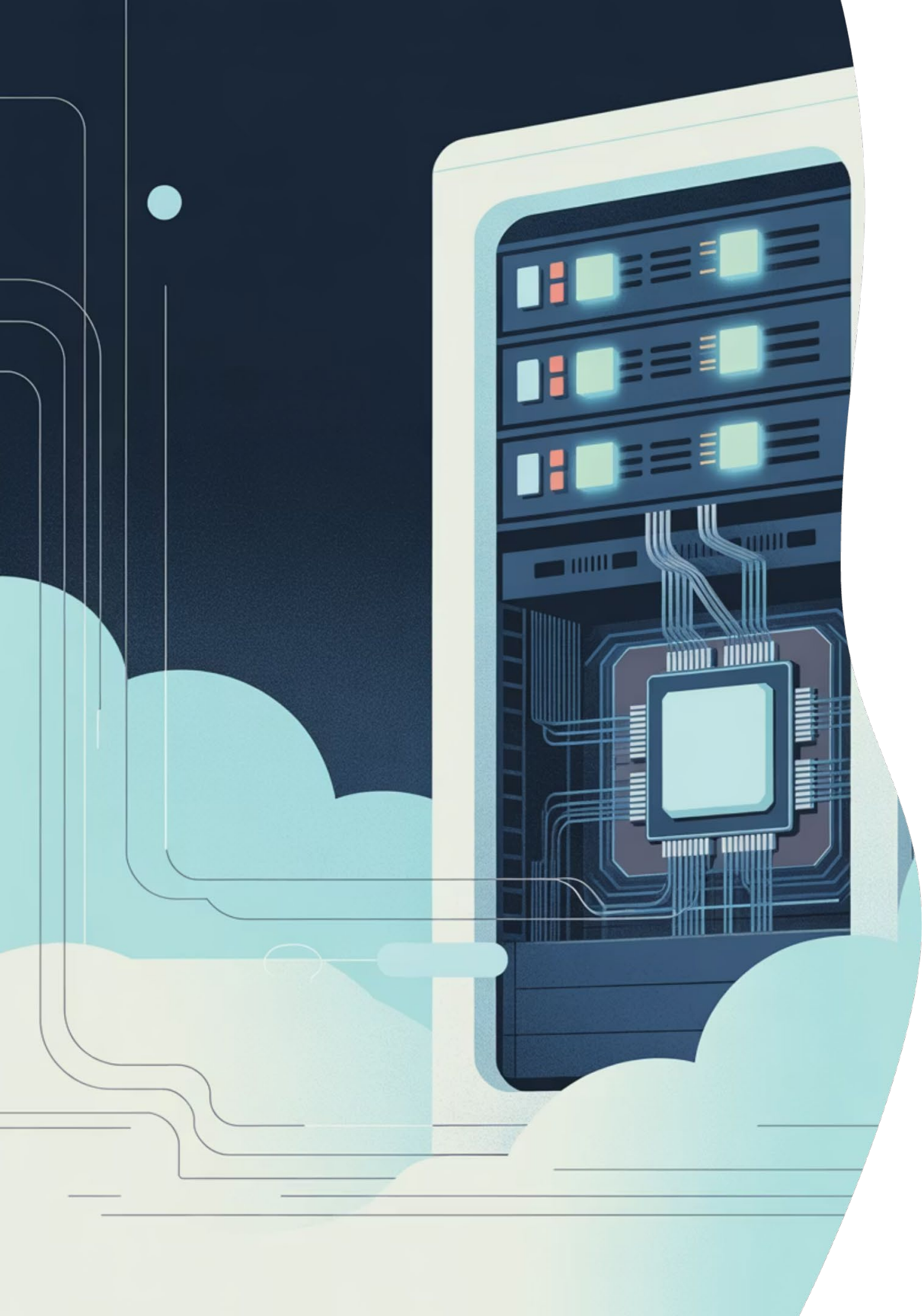
## Estabilidad

Consistencia de respuestas ante entradas similares o repetidas

## Facilidad de integración

Compatibilidad con el stack tecnológico existente

📄 El objetivo final es evaluar la **viabilidad del modelo dentro del sistema completo**, no únicamente el benchmark en tareas aisladas.



# Cuantización en modelos de lenguaje

Los modelos de lenguaje modernos contienen miles de millones de parámetros almacenados como números en coma flotante. Esto implica un consumo de memoria muy elevado. La cuantización es la técnica clave que permite ejecutarlos en hardware modesto.

Modelo 7B · float16

~14 GB de memoria

Modelo 7B · float32

~28 GB de memoria

# ¿Qué es la cuantización?

La cuantización aproxima los valores de los pesos del modelo utilizando **menos bits**, reduciendo drásticamente el tamaño sin sacrificar demasiada calidad.

float16

16 bits por peso — precisión máxima

q8

8 bits por peso — alta calidad

q4

4 bits por peso — equilibrio óptimo

## Ventajas

- Menor uso de memoria RAM y VRAM
- Mayor velocidad de inferencia
- Menor transferencia de datos entre CPU y GPU

## Desventaja principal

- Ligera pérdida de precisión en el modelo



# Cuantización y tamaño de modelo

Gracias a la cuantización es posible ejecutar modelos potentes en portátiles, estaciones de trabajo y ordenadores personales. Este es uno de los factores clave que ha impulsado la popularización de los LLM locales.

14 GB

float16

Modelo 7B sin cuantizar

7 GB

q8

Modelo 7B cuantizado a  
8 bits

3.5 GB

q4

Modelo 7B cuantizado a  
4 bits



# Cómo interpretar el nombre de un modelo

Los modelos descargados con herramientas como **Ollama** incluyen información codificada en su nombre. Saber leerlo permite elegir el modelo adecuado para cada entorno.

📋 **Ejemplo:** `qwen2.5:7b-instruct-q4_K_M`

`qwen2.5`

Familia del modelo

`7b`

~7.000 millones de parámetros

`instruct`

Ajuste fino para seguir instrucciones

`q4_K_M`

Esquema de cuantización de 4 bits, variante K\_M

# Qué significan K y M en la cuantización

Además del nivel de bits, los nombres incluyen letras que identifican **variantes del algoritmo** de cuantización, especialmente en implementaciones como GGUF y llama.cpp.

K

Cuantización basada en **bloques optimizados**. Mejora la distribución de errores frente a la cuantización uniforme estándar.

M

Variante que **preserva mayor precisión** en los pesos más importantes del modelo, minimizando la degradación perceptible.

¿Por qué elegir q4\_K\_M?

La combinación q4\_K\_M ofrece un equilibrio excelente entre:

- Tamaño reducido del modelo
- Alta velocidad de inferencia
- Calidad de respuesta cercana a float16



Es una de las variantes más utilizadas en entornos de producción local.

# Modelos abiertos populares para uso local

El ecosistema open-source ofrece familias de modelos con distintos tamaños y capacidades, adaptados a diferentes necesidades y hardware disponible.



## Qwen

Familia desarrollada por **Alibaba**. Excelente rendimiento multilingüe y en tareas de razonamiento.



## Mistral

Modelos europeos **eficientes y compactos**, con muy buen rendimiento relativo a su tamaño.



## Llama

Familia de **Meta AI**. Ampliamente adoptada, con versiones desde 3B hasta 70B+ parámetros.



## GLM

Desarrollado por **Tsinghua University y Zhipu AI**. Destacado en tareas de comprensión y generación en chino e inglés.

# Modelos recomendados para experimentación

En entornos educativos o de experimentación se priorizan modelos pequeños y equilibrados. La elección correcta depende del hardware disponible.

1

qwen2.5:3b-instruct

Muy ligero y rápido. Ideal para pruebas iniciales.

2

qwen2.5:7b-instruct

Equilibrado. Bueno para la mayoría de tareas.

3

mistral:7b-instruct

Eficiente con buen razonamiento lógico.

4

llama3.1:8b-instruct

Mayor capacidad, requiere más RAM.



# Ejecución de LLMs en local con Ollama

Ollama permite ejecutar modelos de lenguaje open-source directamente en el ordenador local sin necesidad de utilizar servicios en la nube. Esto ofrece varias ventajas:

- privacidad de los datos
- coste cero por inferencia
- menor latencia al evitar llamadas externas
- control completo sobre el modelo utilizado

## Instalación

Descargar e instalar desde:

<https://ollama.com>

Tras la instalación, el servidor se ejecuta automáticamente en segundo plano.

## Descargar y ejecutar un modelo

Ejemplo:

```
ollama run qwen2.5:7b-instruct-q4_K_M
```

Otros modelos recomendados:

```
ollama run qwen2.5:3b-instruct
```

```
ollama run mistral:7b-instruct-q4_K_M
```

```
ollama run llama3.1:8b-instruct-q4_K_M
```

El modelo se descarga automáticamente la primera vez que se ejecuta.

## Uso desde código (API local)

Ollama expone una API REST local.

Ejemplo de endpoint:

```
http://127.0.0.1:11434/api/generate
```

Esto permite integrar el modelo fácilmente desde Python u otros lenguajes.

## Importante: localhost vs 127.0.0.1

En algunos entornos se observa mayor latencia al usar `localhost`.

Esto se debe a que el sistema puede intentar resolver primero IPv6.

Para evitar este problema se recomienda usar directamente:

```
127.0.0.1
```

Ejemplo recomendado:

```
http://127.0.0.1:11434
```

# Ejecución de LLMs en online con la API de *PoliGPT*

```
from openai import OpenAI

client = OpenAI(
    api_key="XXXXXX",
    base_url="https://api.poligpt.upv.es/v1",
)

for model in client.models.list().data:
    print(model.id)
```

|                    |                      |                     |
|--------------------|----------------------|---------------------|
| llama              | mxbai-embed-large    | deepseek-r1:70b     |
| llama-mini         | bge-reranker-v2-m3   | deepseek-r1-coder-  |
| gemma              | granite-docling-258M | tools:32b           |
| phi4               | llama3.1:8b          | qwen3-coder-next    |
| qwen               | llama3.2:3b          | DeepSeek-OCR        |
| qwen-mini          | gemma3:4b            | qwen3-vl:30b        |
| deepseek-reasoner  | gemma3:27b           | llava:7b            |
| deepseek-reasoner- | qwen3:4b             | llama3.2-vision:11b |
| mini               | phi4-mini:3.8b       | llama3.2-vision:90b |
| poligpt            | deepseek-r1:7b       | gemma3-vision:4b    |
| poligpt-vision     | poligpt2             | ollama/*            |
| poligpt-code       | gpt-oss-120b         | tts-1               |
| poligpt-reranker   | llama3.3:70b         | tts-1-hd            |
| poligpt-tts        | qwen3:32b            | kokoro              |
| poligpt-embed      | qwen2.5vl:32b        | qwen3-tts           |
| bge-m3             | qwq                  | ollama/llama2       |
| nomic-embed-text   | deepseek-r1:32b      |                     |

```
from openai import OpenAI

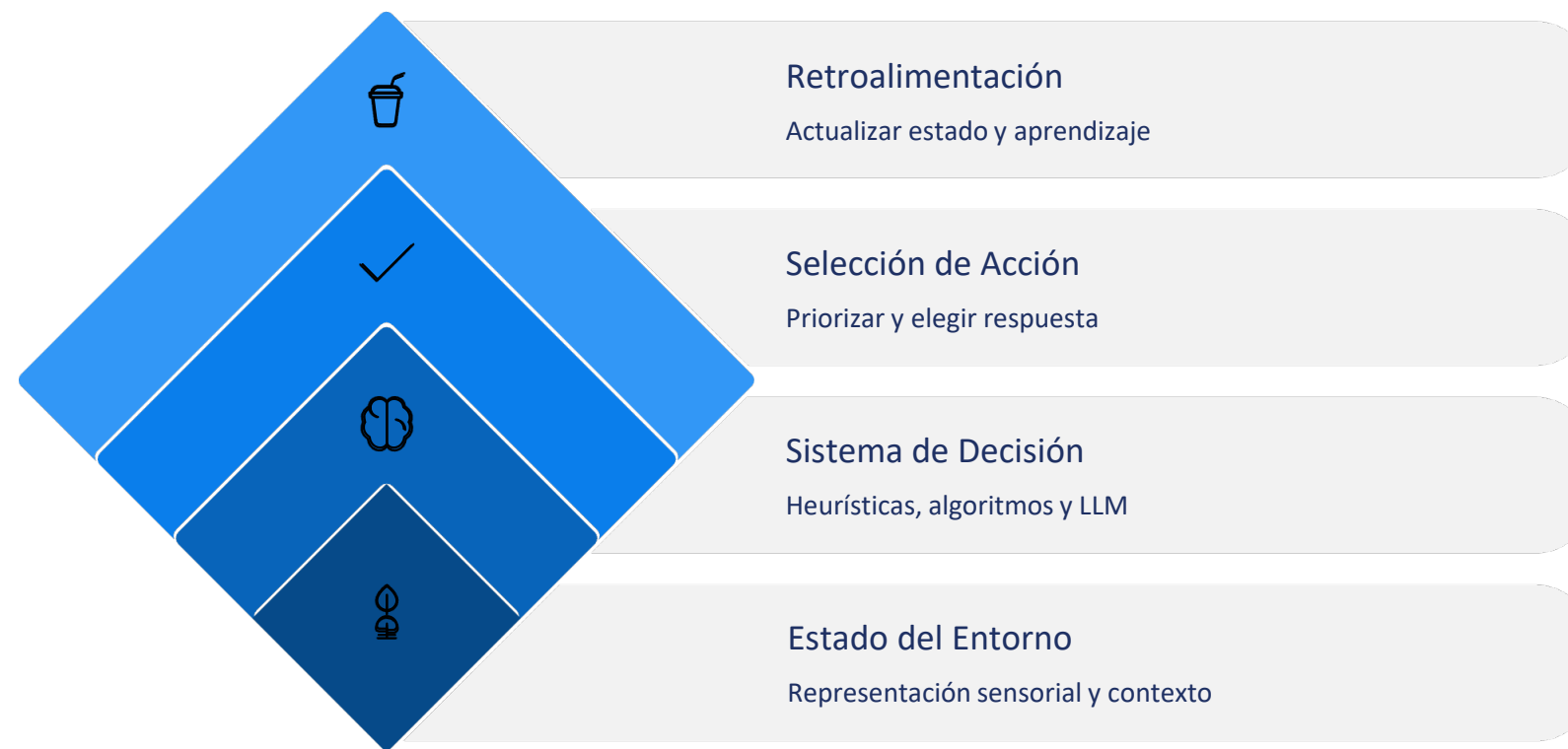
client = OpenAI(
    api_key="XXXXXX",
    base_url="https://api.poligpt.upv.es/v1",
)

response = client.chat.completions.create(
    model="llama-mini",
    messages=[
        {"role": "system", "content": "Eres un asistente útil."},
        {"role": "user", "content": "Curiosidad sobre Valencia."}
    ],
    temperature=0.7, # ajusta la creatividad de la respuesta
    (Número bajo = más precisa, Número alto = más creativa)
)

print(response.choices[0].message.content)
```

# Arquitecturas híbridas en sistemas inteligentes

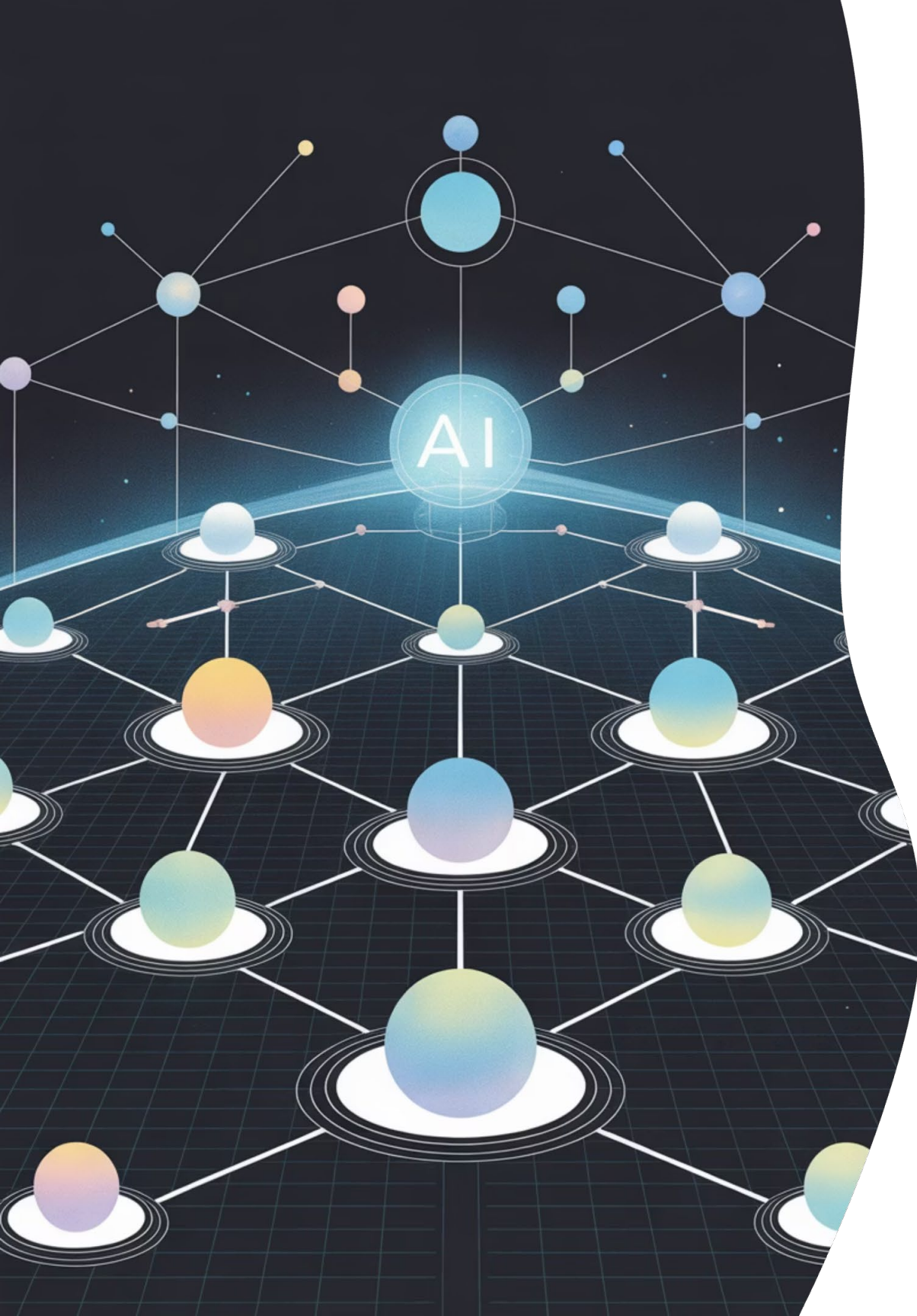
En sistemas complejos, los LLM no actúan solos. Se integran como **un componente más** dentro de una arquitectura que combina distintos métodos de decisión.



## ¿Por qué arquitecturas híbridas?

- **Heurísticas:** decisiones rápidas y deterministas
- **Algoritmos clásicos:** optimización y búsqueda eficiente
- **LLM:** razonamiento flexible y comprensión contextual

La combinación permite sistemas más robustos y adaptativos que cualquier método aislado.



# El LLM como componente dentro de sistemas

Los Large Language Models son una herramienta **poderosa pero complementaria**. En sistemas inteligentes reales, se combinan con otros métodos para maximizar capacidad, velocidad y robustez.



## Heurísticas

Velocidad y reglas deterministas



## Algoritmos clásicos

Optimización y búsqueda estructurada



## Machine Learning

Aprendizaje a partir de datos



## LLM

Razonamiento y comprensión contextual



# Reflexión Final

Los LLMs aportan capacidades de razonamiento y análisis a sistemas complejos, pero su integración introduce consideraciones que deben gestionarse con rigor técnico.

## Factores a considerar

### Latencia y coste

Cada llamada tiene un coste temporal y económico no despreciable

### Diseño de prompts

La ingeniería de prompts es una disciplina crítica en la producción

### Integración modular

El LLM debe encajar limpiamente con el resto de componentes

## La solución más efectiva

En la mayoría de sistemas inteligentes, la estrategia óptima combina:

- **Heurísticas rápidas** para decisiones frecuentes y de baja complejidad
- **LLMs** para decisiones estratégicas que requieren razonamiento contextual

